

Acelerador em FPGA do Supervisor LiDAR

Co-Projeto HW–SW sobre Zynq-7000

Cadeira de rodas semi-assistida

Vítor Gonçalves de Andrade Silva Thiago Henrique Oliveira Souza

2026/1

Engenharia Eletrônica — FCTE/UnB

O sistema

Cadeira de rodas motorizada **semi-assistida**.

Princípio de segurança:

o sistema pode **FREAR** ou **DESVIAR**,
mas **nunca ACELERAR**.

A iniciativa é sempre do usuário.

- Raspberry Pi 5 + ROS 2 (percepção)
- ESP32-S3 (tempo real: joystick, motores, encoders)
- RPLIDAR C1 a 10 Hz, 720 feixes

O supervisor

A cada varredura, decide se o avanço comandado deve ser **reduzido** ou **interrompido**.

Por que acelerar isto?

É o nó mais pesado e o único cuja latência tem **consequência física**: ela vira **centímetros** rumo ao obstáculo.

O algoritmo: teste de corredor

Projeta cada feixe no referencial do veículo:

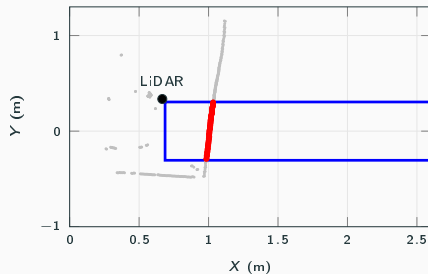
$$X = x_L + r \cos(a + \theta_L)$$

$$Y = y_L + r \sin(a + \theta_L)$$

É obstáculo se cair no **corredor** que a **cadeira vai ocupar**:

$$|Y| \leq W/2 \quad \text{e} \quad X > X_f$$

Para cada um dos $K = 19$ candidatos de desvio (-45° a $+45^\circ$), gira-se o ponto e repete-se o teste.



Varredura real da cadeira. Vermelho: no corredor.

$$N \times K = 720 \times 19 = 9.918 \text{ avaliações por varredura}$$

Por que este bloco: um defeito real de segurança

O supervisor **antigo** media a folga com um **cone** ancorado no sensor.

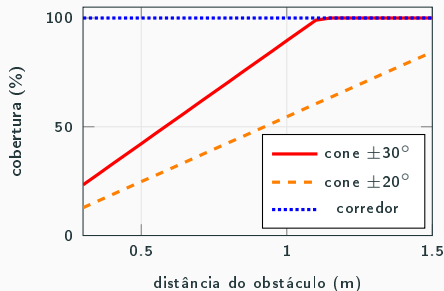
Só valeria se o LiDAR estivesse **no eixo** da cadeira.

Ele está **33,6 cm fora do eixo**.

Consequência

A metade **direita** da cadeira ficava **cega** — e a cobertura **piora** quanto mais **perto** está o obstáculo.

A cadeira **atropelava** obstáculos à direita.



A 0,60 m (zona de parada) o cone vê só 52 %.

Etapa	Custo	Onde
Projeção + teste de corredor	$O(N \cdot K) = 9.918$	FPGA
Função custo + <i>argmin</i>	$O(K) = 19$	ARM

Por que *não* levar tudo para hardware?

A função custo são **19 comparações** — o ganho seria irrisório.

Em compensação, é nela que moram os **pesos ajustados em campo** (w_{obs} , w_{dev} , K_{assist}), retocados a cada ensaio.

Congelá-los em RTL trocaria flexibilidade por nada.

É exatamente o *trade-off* que o co-projeto existe para arbitrar.



CORDIC compartilhado ($O(N)$)

- Só somadores e *shifts* $\Rightarrow$ **zero DSP**
- *Por que não LUT de seno?* O ângulo depende da **calibração**, reescrita em tempo de execução via AXI4-Lite.

19 pistas replicadas ($O(N \cdot K)$)

- $\cos \delta_k, \sin \delta_k$ são **constantes de síntese**
- $\Rightarrow$ 4 multiplicadores fixos por pista
- Avaliam **no mesmo ciclo** o que o software **itera**

Vazão: 1 feixe/ciclo, sem bolhas.

Verificação: equivalência bit a bit

1. Modelo **ponto fixo** (Python) emula o RTL *exatamente*
2. Modelo **float** = referência física
3. O modelo gera **estímulo** e **esperado**
4. Testbench VHDL lê os arquivos e compara as 19 folgas

Detalhe que evita um erro invisível

As constantes do RTL (`cordic_pkg.vhd`) são **geradas** pelo mesmo modelo. RTL e referência casam **por construção** — não por transcrição manual.

Resultado (GHDL)

```
varredura 0: EQUIVALENCIA OK
varredura 1: EQUIVALENCIA OK
varredura 2: EQUIVALENCIA OK
```

Zero divergência

O estímulo **não é sintético**: são três varreduras **reais** da cadeira, com um balde a 21 cm à direita — o antigo ponto cego.

O primeiro modelo em ponto fixo errava 402 mm.

Não era arredondamento. Era **estouro de faixa**:

$$\theta = a_{\min} + i \Delta a + \theta_L \rightarrow -5,85 \text{ rad}$$

Q3.13 comporta apenas $\pm 4,0$ rad

O ângulo **saturava** $\Rightarrow$ o feixe era projetado numa direção completamente errada.

Correção: envolver θ em $[-\pi, \pi)$.

	médio	máximo
θ saturado	17,89 mm	402 mm
θ envolvido	0,99 mm	5,27 mm

LSB de Q6.10 = 0,98 mm

$\Rightarrow$ erro final ≈ 1 LSB

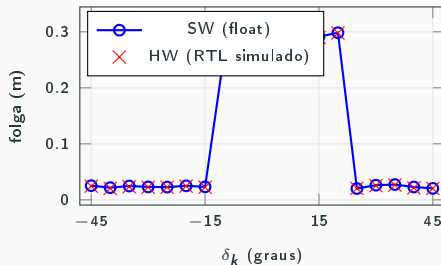
$\Rightarrow 0,9\%$ da zona de parada

Lição

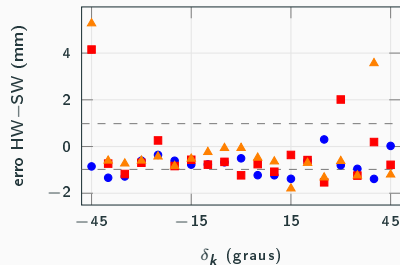
Em ponto fixo, a **FAIXA** costuma ser mais crítica que a **RESOLUÇÃO**.

Perdemos 402 mm por 3 bits de parte inteira — não por falta de bits fracionários.

Saída do HW vs saída do SW



As curvas se sobrepõem — é esse o resultado.

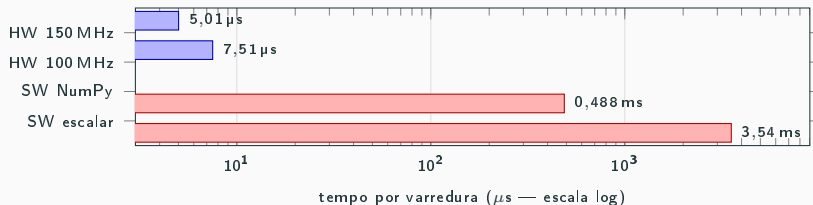


Tracejado: ± 1 LSB. 3 varreduras, 57 comparações.

Erro médio 0,99 mm | máximo 5,27 mm | ≈ 1 LSB | 0,9% da zona de parada

O “HW” é a saída real da simulação VHDL (`rtl_out`), não o modelo.

Implementação	Tempo/varredura	Speedup	
		vs escalar	vs NumPy
Software escalar (Pi 5)	3,54 ms	—	—
Software NumPy (Pi 5)	0,488 ms	7,3×	—
HW 100 MHz (751 ciclos)	7,51 μs	472×	65×
HW 150 MHz	5,01 μ s	707×	98×



Então qual é o valor real?

1. Alívio de CPU

Libera 8,3 % numa Pi que também roda SLAM e EKF — e que já **congelou** por exaustão de memória.

2. Latência determinística ← o mais importante

Um nó Python está sujeito ao escalonador de um Linux não-preemptivo. Num laço de **segurança**, determinismo vale mais que média baixa.

3. Orçamento para crescer

Dá para aumentar K ou a densidade angular sem custo de tempo: o gargalo passa a ser o **sensor**.

Reportamos isso explicitamente porque a leitura oposta — “aceleramos, logo melhorou” — seria **desonesta com os dados**.

	prev.	obtido	%
LUT	—	2.445	4,6
FF	—	1.808	1,7
DSP48	78	54	24,5
BRAM	0	0	0

Timing e potência

WNS = 0,872 ns $\Rightarrow$ **fecha a 100 MHz**

$F_{\max} = 109,6$ MHz

Potência: 0,246 W

(dinâmica 0,140 W + estática 0,106 W)

0 erros, 0 *critical warnings*

Erramos a previsão de DSP — para MAIS

Previmos **78** ($19 \times 4 + 2$). A ferramenta usou **54** — 31 % a menos.

Por quê? O DSP48E1 não é um multiplicador isolado: é um **multiplicador-acumulador** com porta C e cascata PCIN.

A síntese reconheceu que

$$x^{(k)} = X \cos \delta_k + Y \sin \delta_k$$

é a forma que o bloco implementa **nativamente**, e fundiu produto + soma num *único* DSP.

Contar operadores do RTL **superestima** o custo: o mapeamento explora estrutura do dispositivo que não aparece no código.

O que a síntese confirmou

CORDIC: 0 DSP

Os **54** DSP estão *todos* no `corridor_core` (pistas + projeção).

O CORDIC usa só somadores e deslocamentos cabeados — exatamente o que se pretendia ao escolhê-lo.

BRAM: 0

A arquitetura *streaming* não precisa armazenar a varredura: cada feixe é consumido **no ciclo em que chega**.

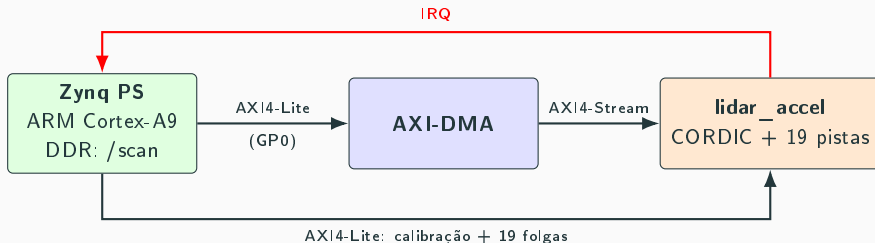
Folga para crescer

Sobram 75 % dos DSP.

Dá para **triplicar** o número de candidatos K sem sair do xc7z020 — ou aumentar a precisão das pistas.

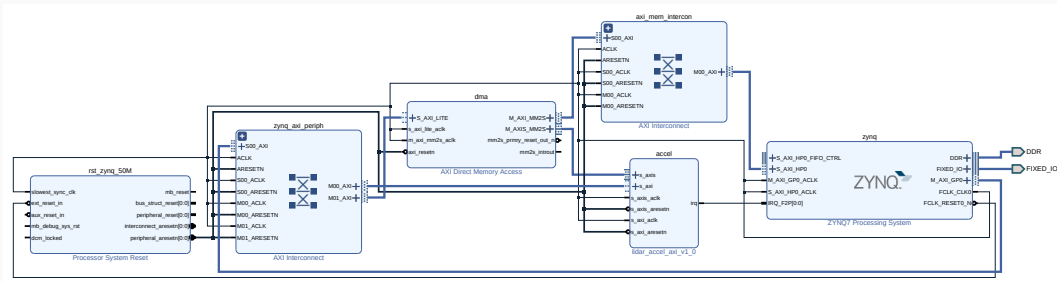
E o gargalo continuaria sendo o **sensor** (100 ms), não a computação.

Block design: ARM + AXI-DMA + acelerador



- O ARM guarda a varredura na **DDR**; o **AXI-DMA** a transmite como *stream* ao acelerador (via S_AXI_HP0)
- Resultado (19 folgas) volta por **AXI4-Lite**; a **IRQ** avisa o fim — sem espera ocupada
- *Stream* para o fluxo volumoso; mapeado em memória para os escalares.

Block design no Vivado



zynq → zynq_axi_periph (AXI4-Lite) → dma e accel | dma/M_AXIS_MM2S → accel/s_axis (AXI4-Stream) | accel/irq → IRQ_F2P

O que foi alcançado

472× vs software escalar, 65× vs vetorizado, com equivalência **bit a bit** verificada sobre **dados reais** da cadeira.

Análise crítica — esperado vs alcançado

O *speedup* veio, mas o **ganho de tempo não era necessário**: o software já cabia no orçamento do sensor. O benefício legítimo é o **alívio de CPU** e, sobretudo, o **determinismo** da latência num laço de segurança.

O resultado mais transferível

Não é o *speedup*: é o **defeito de faixa em Q3.13**. Um ângulo de $-5,85$ rad saturando num formato de ± 4 rad produziu 402 mm de erro **silencioso** — o RTL não acusa, a simulação “roda”.

Foi o *golden model*, não a inspeção de ondas, que o encontrou.

Obrigado!

fpga/ — RTL, golden model, testbench, scripts Vivado

```
bash sim/run_ghdl.sh
```

reproduz a prova de equivalência em um comando

Backup: formato de ponto fixo

Grandeza	Formato	Faixa	LSB
distâncias, coords.	Q6.10	± 32 m	0,98 mm
cos, sin	Q1.14	± 2	$6,1 \times 10^{-5}$
ângulos	Q3.13	$\pm 4,0$ rad	$1,22 \times 10^{-4}$ rad

A faixa de Q3.13 (± 4 rad) foi a origem do defeito de 402 mm: θ chega a $-5,85$ rad.

Backup: por que CORDIC e não LUT?

Uma LUT de seno exigiria uma ROM indexada pelo **índice do feixe**.

Isso só funcionaria se `angle_min`, o incremento e o `yaw` fossem **fixos em síntese**.

Mas eles são **parâmetros de calibração** — reescritos em tempo de execução por AXI4-Lite sempre que o LiDAR é remontado.

CORDIC

- Aceita **qualquer** ângulo em tempo de execução
- Só somadores e deslocamentos cabeados $\Rightarrow$ **zero DSP48**
- 16 estágios pipelinados $\Rightarrow$ 1 resultado/ciclo
- Redução de quadrante na entrada (o laço só converge para $|z| \leq 1,7434$ rad)